End-to-End SLA Estimation from Limited Provider Disclosures

Methods, Experimental Record, and Limits of Reconstruction

Ildefons Magrans de Abril Xavier Franch Schahram Dustdar

16 October 2026

Internal technical report, first typeset draft. Not approved for public release.

Contents

Executive synthesis 2

1. Research problem, scope, and prior context 3

2. Formal SLA objective and benchmark semantics 4

3. Provider disclosures 5

4. Integration algorithms and historical method versions 6

5. Native simulator, graph structures, and experiment protocol 11

6. Controlled pilot (Phase I) 13

7. Expanded evaluation (Phase II) 15

8. Retrospective investigation (Phase III) 17

9. Uncertainty mathematics and cross-experiment interpretation 21

10. Threats to validity and reproducibility 21

11. Conclusions and falsifiable next questions 21

Appendix A. Mathematical and numerical interpretation 21

Appendix B. Archived source and audit provenance 21

Appendix C. Experimental chronology and method versions 21

Appendix D. Provisional literature references 21

Executive synthesis

In composed cloud-edge services, individual providers may advertise limited workload and SLA evidence while keeping their stochastic execution models private. The methodological challenge is to combine these separate disclosures into a prediction of end-to-end SLA satisfaction over a time horizon, without assuming access to hidden process parameters. We report the development, implementation, and evaluation of a family of methods indexed by provider information representation I and integration procedure M, together with a simulator and an account of observed limitations. This internal report records the experiments and their evidential limits. It does not propose a general validation methodology for management or control algorithms.

We define the evaluated quantity as $\sigma_G(A, H; \rho) = \Pr[c_G(A, H) \geq \rho]$, where $c_G(A, H)$ is the fraction of **decided** composed requests that meet a specified latency-cost-quality region by time H. A missed latency deadline is a decided failure; requests not yet decided are excluded; zero decided requests yield compliance one. This quantity measures the probability that the composed service meets its SLA compliance target over the observation horizon H.

First, the controlled pilot (Phase I) tested the original three-provider parallel graph. The original controlled pilot was promising. Against a fresh white-box reference ($N_{\mathrm{WB}} = 200$: 200 new simulation runs using the true underlying model), across 555 related measurements (15 SLA conditions, each evaluated at 37 time horizons using shared simulation runs), the mean absolute errors (MAE) of a single reconstructed model (M1), a 27-member equal-weight ensemble (M2), and a Gibbs-weighted pilot M3 with 14 propagated joint models were respectively **0.18517, 0.06815 and 0.03037**. However, a retrospective cost-matched support audit found a Top1 estimator with MAE **0.02878**, slightly below the weighted Top14 estimator in that fixed pilot. The pilot therefore did not establish that multi-model averaging was necessary.

Next, the expanded evaluation (Phase II) evaluated generalization across multiple provider configurations and composition graphs. Sixteen evaluation cases were planned (four provider configurations combined with four composition graphs). The four cases involving configuration P2 did not meet the predefined criteria for informative SLA test conditions, so they were excluded without replacement. On the remaining 12 cells, with independent $N_{\mathrm{WB}} = 1000$ final white-box trajectories per cell and 6,660 related measurements (555 per case) for each of M1–M3, the executed Phase-5 V3b algorithms produced MAE **0.15702** (M1), **0.13395** (M2), and **0.12189** (M3 FULL343). M3 retained the best aggregate ranking, but hard conditions still had substantial errors. Its algorithm and candidate support also changed from the pilot and from the initially registered Phase-5 method. Accordingly these numbers cannot be treated as direct replications of the original 14-member method.

Finally, the retrospective investigation (Phase III) investigates that gap, including an original-style M3 comparison, without creating a new independent holdout. Reconstructing 48 public-I1-scored candidates/provider and propagating Top14 at B=1400 on the same 12 cells gave **MAE 0.15416** (separately simulated Top1, **0.16228**), versus **0.12189** for executed V3b FULL343. Although Top14 retained at least **99.64%** of inferred joint-model weight in P1, P3 and P4, FULL343 outperformed it in **11 of 12** cells. These post-result comparisons do not isolate member count or constitute a fresh holdout.

Subsequent development tests revealed further limitations. On P4/G_SEQPAR, stronger reconstruction from I1 alone performed comparably to, and on average slightly better than, several richer I2 information-and-loss pipelines under matched search and graph-propagation designs. This does **not** isolate the causal utility of information quantity because fitting objectives also changed. In a separate P2/G_PAR experiment using a deterministically composed public graph region, M1, M2 and M3 achieved MAE **0.13200, 0.14058 and 0.12787**, respectively, under **unmatched** graph budgets. A privileged retrospective audit then found that the true generating provider parameters fitted the public I1 evidence better than the selected reconstructed candidates despite lying inside the search domain. This is direct evidence of a search/reconstruction deficit, not a proof of structural non-identifiability.

The overall conclusion is conditional: end-to-end SLA estimation from limited disclosure is demonstrably feasible **in some controlled cases**, but the developed finite-search integration methods have not delivered reliable prediction across the studied variations. This report preserves favorable, negative, and inconclusive findings, including differences between the single-graph pilot and the broader battery, incomplete Step-0 eligibility, unequal simulation budgets, and limits of the reported uncertainty measures. These observations provide **initial empirical motivation**, not a validated procedure, for a separate methodological paper on evaluating management, control, and optimization algorithms in hierarchical computing-continuum systems. That future work is intended to address how to select evaluation graphs and scenarios, choose simulation sample sizes, and estimate errors and uncertainties before making broader performance claims.

1. Research problem, scope, and prior context

1.1 Composable offerings and information boundaries

The computing continuum spans resources and services controlled by different administrative entities. An integrator may know its own workflow or service-composition graph while observing only restricted public claims about the providers on which it depends. Physical execution is shaped by stochastic service times, queues, branch synchronization, workload and cost accumulation. Predicting the performance of composed black-box services using only their providers' advertised performance measures is not equivalent to observing the resulting end-to-end behavior.

More generally, empirical claims about management and control algorithms for hierarchical computing-continuum systems depend on the selected graph structures, workload and provider conditions, simulation budgets, and reference measurements. This report studies one specific prediction problem and documents what its experiments establish. The broader question of how to design such evaluations systematically is reserved for future methodological work.

The Composable Autonomic Offering (CAO) abstraction represents an offering as

$$o = (G, s, \sigma, r), \tag{1}$$

where G is the composition graph, s describes service objectives, σ represents dependability or survival information, and r is the realization envelope [12]. The CAO abstraction motivates predicting a useful dependability quantity, but does not, by itself, prescribe which information must be disclosed or which numerical method can produce a dependable composition estimate.

We separate these choices explicitly:

$$\tau = (I, M), \tag{2}$$

with **information interface** I and **integration method** M. The interface I specifies what each provider discloses. Given these separate disclosures and the known composition graph G, the integrator applies M to estimate the composed service's end-to-end SLA probability, $\widehat{\sigma}_G(A, H; \rho)$. Depending on the method, this may involve reconstructing provider models and propagating their predictions through G. Distinct M may use the same I; conversely an information-rich interface can require a different reconstruction objective, making the evaluated unit an **information–method pair**.

The question is not whether analytical or stochastic QoS composition exists. It does [3,8,17–19]. Rather, the specific problem is **inverse, public-evidence-constrained** composition: can an integrator combine the probability surfaces advertised by individual black-box providers to estimate the composed service's finite-horizon, end-to-end SLA probability, without access to their execution traces or true parameters?

1.2 Research questions and evidence categories

The program addresses four related questions:

RQ1, feasibility and limits. When can finite public provider disclosures support useful graph-level SLA-satisfaction prediction, and in which worlds, topologies, regimes, or query regions does prediction fail?

RQ2, methods under a fixed interface. How do analytical propagation, a single provider reconstruction, equal-weight reconstruction portfolios and evidence-weighted finite supports compare? Which observed differences are due to candidate search, support membership, weights or numerical allocation, and which cannot be causally separated?

RQ3, disclosure. Does enriching the public interface with marginal or paired temporal compliance information improve the resulting *information–loss–integration pipeline* under matched reconstruction resources?

RQ4, provenance and decision meaning. How reproducible are the finite results, and what do numerical Monte Carlo intervals, model spread, reference error and support-truncation guarantees actually mean for an SLA threshold?

We classify the evidence by its scientific role, rather than by the names of repository folders or successive software versions. **First, a controlled pilot (Phase I)** tested the methods in a single provider world and graph; it established a promising initial result, not cross-setting reliability. **Next, the expanded evaluation (Phase II)** tested multiple worlds and composition graphs using a final white-box reference held back until predictions were frozen. Its implementation was revised during development, so this is not an unchanged confirmation of the pilot method. **Finally, retrospective diagnosis and further development (Phase III)** compared historical variants, investigated divergence and explored richer provider disclosures after earlier results were known. These later tests do not constitute another independent holdout. Appendix C provides the experiment-by-experiment chronology and exact method-version identifiers.

1.3 Scientific boundaries

This document does not establish a structural identifiability theorem. Multiple candidate models with different parameters and similar *finite empirical* I1 errors may reflect a non-injective information map, measurement noise, search limitations, model family restrictions, or some combination. The present observations cannot distinguish these causes in full generality.

The report is an **empirical record**, not a proposed or validated general-purpose experimental protocol. Its findings apply to the particular provider worlds, graph structures, query-selection rules, budgets and reference procedures actually tested. They motivate further methodological work but do not determine which graphs, sample sizes or uncertainty estimators should be used to validate other hierarchical management and control algorithms.

2. Formal SLA objective and benchmark semantics

2.1 Request-level admissibility

For a composed request, denote latency, accumulated cost and quality by (L_G, C_G, Q_G). A rectangular admissibility region is

$$A = \{(L, C, Q) : L \le \ell,\ C \le c,\ Q \ge q\}.$$

A request is **compliant** when it is decided and its final outcome lies in A. The simulator records the top-level emission and, if available before simulation stop, the completion time. An on-time request is decided at completion. An uncompleted or late request is decided as a latency failure when its deadline arrives. A request whose completion and deadline are still in the future is unresolved at the current evaluation horizon. The SLA-scoring rule does not equate "unresolved" with "failed".

Let $\mathcal{D}_G(H)$ be the set of top-level requests *decided* by horizon H, and $\mathcal{C}_G(A, H) \subseteq \mathcal{D}_G(H)$ the compliant subset. Define, with the explicitly frozen zero-decision convention,

$$c_G(A, H) = \begin{cases} 1, & |\mathcal{D}_G(H)| = 0, \\ |\mathcal{C}_G(A, H)|/|\mathcal{D}_G(H)|, & |\mathcal{D}_G(H)| > 0. \end{cases} \tag{3}$$

For required compliance fraction $\rho \in [0, 1]$, the true graph-level estimand is

$$\boxed{\sigma_G(A, H; \rho) = \Pr\{c_G(A, H) \ge \rho\}.} \tag{4}$$

The probability is taken over independent stochastic trajectories of the specified provider world, workload and execution graph. The horizon H determines which decided requests are included in the compliance fraction. As more requests are decided, this fraction may rise or fall, so $\sigma_G(A, H; \rho)$ is **not guaranteed to decrease with** H. We therefore use the precise term "horizon-dependent SLA-satisfaction probability" throughout the report.

The numerical methods compute $\widehat{\sigma}_G^{(I,M)}(A, H; \rho)$, and the hidden world is only evaluated through a separate finite white-box reference $\widehat{\sigma}_G^{\mathrm{WB}}$ under experimental conditions that permit it.

2.2 Prediction target and evaluation reference

What do we estimate? An SLA prediction query $(A, H; \rho)$ specifies an admissibility region A (latency, cost and quality limits), an observation horizon H, and a required fraction ρ of decided requests that must comply with A by H. For a known composition graph G, Equation (4) defines $\sigma_G(A, H; \rho)$, the probability that the composed service meets this requirement. The integrator does not know the providers' hidden stochastic execution models. It has only the information each provider discloses through interface I. Integration method M uses those disclosures and the known graph to produce the end-to-end prediction $\widehat{\sigma}_G^{(I,M)}(A, H; \rho)$. **The scientific task is to assess the accuracy and limits of this probability prediction.**

How do we evaluate it? In the simulator, the true provider models can be used to generate an independent finite white-box reference $\widehat{\sigma}_G^{\mathrm{WB}}(A, H; \rho)$. We compare each method's prediction with this reference across the tested SLA queries, horizons, provider worlds and graph topologies. The hidden provider models used for final reference generation are not supplied to the methods being evaluated. In the results tables, a *scored prediction query* is a complete query $(A, H; \rho)$ for which a method returns a numerical probability estimate and is compared with the white-box reference. Queries without a numerical prediction are not included in that method's error average. Because the reference is computed from a finite number of stochastic runs, it is itself an estimate of σ_G, not an exact value. Numerical simulation error and reconstruction error must therefore be distinguished (Section 9 and Appendix A).

3. Provider disclosures

3.1 Public interface I1

Under the original public interface I1, each provider advertises a finite family of rectangular admissibility regions indexed by a target content level ρ_r:

$$A_i(\rho_r) = \{L_i \le \ell_i(\rho_r),\, C_i \le c_i(\rho_r),\, Q_i \ge q_i\}. \tag{5}$$

Here L_i, C_i and Q_i are the provider's latency, cost and quality outcomes. The functions $\ell_i(\rho_r)$ and $c_i(\rho_r)$ specify **numerical latency and cost limits** for each advertised index ρ_r. They are finite lookup mappings, not time-varying performance functions. In this simulated experiment, the framework prepares a separate I1 card on behalf of each provider and makes it available to the integrator. To construct its advertised regions, the provider-side procedure fits a joint model of private latency and cost observations (a Gaussian mixture in log latency and log cost). It draws a fixed synthetic sample from that model and, for each ρ_r, chooses a minimum-area rectangle containing at least that fraction of sampled outcomes, while retaining the preceding region. Consequently the regions are **nested**: when $\rho_r < \rho_r'$, $A_i(\rho_r) \subseteq A_i(\rho_r')$, so neither the latency nor the cost limit decreases. In the studied provider worlds, the quality outcome is constant at $Q_i = 0.5$, with advertised limit $q_i = 0.5$. Thus quality is not a varied experimental dimension, and the regions differ only in their latency and cost limits.

The index ρ_r thus specifies the **target content of the fitted latency-cost model**, not a guaranteed fraction of compliant requests during an execution horizon. The latter is the separate SLA-compliance threshold ρ_q. Once the regions are fixed, each provider advertises an empirical probability surface

$$\widehat{\sigma}_i^{I1}(A_i(\rho_r), H; \rho_q) \approx \Pr\{c_i(A_i(\rho_r), H) \ge \rho_q\}. \tag{6}$$

The historical pilot used $\rho_r, \rho_q \in \{0.95, 0.975, 0.98333, 0.99, 0.995\}$ and a 5-second horizon grid up to 240 seconds, with a workload period of 0.2 seconds. Provider-card acquisition used two disjoint banks of 100 complete simulation trajectories each. The first bank (seeds 6000–6099) was used only to fit the provider's latency-cost model and construct the regions. The second bank (seeds 6100–6199) evaluated the already fixed regions. For each $(A_i(\rho_r), H, \rho_q)$, each trajectory in the second bank contributed one binary outcome: whether its decided-request compliance fraction reached ρ_q by horizon H. Thus $\widehat{\sigma}_i^{I1} = n_{\mathrm{pass}}/100$ is the fraction of successful trajectories, **not** an average of raw traces, the mean of per-trajectory compliance fractions, or the fraction of compliant requests pooled across trajectories. The same trajectories are reused across regions, horizons and compliance thresholds, so the points on the probability surface are correlated. Each simulation run also supplies provider-local ledgers for all three providers.

The public I1 card includes pointwise 95% Wilson binomial confidence intervals for each estimated SLA-satisfaction probability (the chance that a new independent provider trajectory achieves at least the required fraction ρ_q of compliant decided requests for region $A_i(\rho_r)$ by horizon H). These intervals are disclosed as part of I1, but none of the evaluated methods M0, M1, M2 or M3 uses them. The card does **not** disclose request timestamps, hidden queue/service trajectories or provider parameters.

The provider-side procedure uses private simulation evidence to construct the finite I1 card. The integrator, by design, sees only that card. This provider-versus-integrator information boundary is essential: giving a method the hidden sample identities, generating parameters or white-box graph predictions would change I and invalidate the intended inverse reconstruction comparison.

Note that I1 discloses provider-level regions A_i, not the graph-level SLA query region A_G for the composed service. For each experimental comparison, A_G is fixed and shared by the methods being evaluated.

3.2 Enriched I2 interfaces

The post-result I1/I2 experiments were developed in the repository's `first_science/phase6/` folder (historical Phase-6) and belong to global Phase III in this report. They retain the I1 provider regions and horizon grid and reuse the corresponding provider-local simulation evidence. What changes is the information disclosed to the integrator about how often, and how consistently, each provider satisfies its advertised regions.

I2a: distribution of compliance across simulation runs. Fix one advertised provider region and one evaluation horizon. In each independent simulation run, the provider handles many requests. We calculate the percentage of requests decided by that horizon that satisfy the region's latency, cost and quality limits. This gives **one compliance percentage per run**. To illustrate, suppose four runs produce 98%, 96%, 94% and 70% compliance. If the SLA requires at least 95%, two runs satisfy it and two do not. For this threshold, I1 would report only the pass rate, 2 out of 4 runs (50%). I2a would instead disclose the four compliance percentages themselves, sorted as 70%, 94%, 96% and 98%. The integrator can therefore see not only how often the SLA is met, but also how much compliance varies and how severely it can fall below the requirement. In the actual experiment, **100 independent runs**, rather than four, provide 100 compliance percentages for each advertised region and horizon. These percentages are sorted separately for every horizon, so I2a does not reveal which values at different horizons came from the same run.

I2b: paired compliance at two horizons. I2b includes the I2a distributions and adds pairs of compliance fractions measured in the *same* trajectory at two selected horizons. A pair such as (0.98, 0.91) means that cumulative compliance in one run declined from 98% to 91% between those horizons; the fractions still refer to all requests decided since the start, not just requests processed within the interval. For each selected interval, the published representation contains 100 such pairs. The frozen design samples intervals starting at 5 seconds and then every 10 seconds, with separations of 20, 30, 40, 50, 75, 100, 150 or 200 seconds where both endpoints fit within the 240-second horizon. The pairs reveal whether good or poor compliance tends to persist or change over time. They contain no trajectory identifiers, and the published records cannot be linked across different intervals by a shared identifier; they are not complete request histories or labelled trajectories.

Additional disclosure in I2b-v4. The later I2b-v4 extension publishes one more value for each selected interval: the average, across the 100 runs, of the difference between the highest and lowest cumulative compliance fractions recorded during that interval on the 5-second evaluation grid. This describes variation *inside* an interval that its two endpoint values alone cannot reveal. Individual runs' ranges and complete histories remain private. Section 4.3 explains how M2 uses I1, I2a, I2b and this additional summary to reconstruct provider models. The results appear in Section 8.3.

4. Integration algorithms and historical method versions

4.1 M0: analytical public-boundary predictor

M0 asks a worst-case question: **If every provider meets its advertised limits for a request, what limits follow for the complete service?** The advertised region $A_i(\rho_r)$ (Eq. (5)) specifies maximum latency ℓ_i, maximum cost c_i and minimum quality q_i. In a sequence, latencies add. In an all-required parallel join whose branches start together, latency is determined by the slowest branch. Cost adds because

every required service incurs cost. The model defines composed quality as the minimum across required services. Before including processing and communication overheads, the limits are

$$\begin{aligned} \text{Seq}: &\quad \ell = \sum_i \ell_i, \quad c = \sum_i c_i, \quad q = \min_i q_i, \\ \text{ParAll}: &\quad \ell = \max_i \ell_i, \quad c = \sum_i c_i, \quad q = \min_i q_i. \end{aligned} \tag{7}$$

These limits apply to a request only if all required providers respect their advertised limits. M0 applies the same rules recursively to graphs built from sequence and all-required parallel operations. For example, in $G = A \to (B \parallel C)$, the provider-only latency limit is $\ell_A + \max(\ell_B, \ell_C)$, cost is $c_A + c_B + c_C$, and quality is $\min(q_A, q_B, q_C)$. Fixed processing, network and synchronization costs or delays are included at their actual positions in the graph.

For the frozen Phase-I pilot, Source $\to$ Fpre $\to (A \parallel B \parallel C) \to$ Fpost, three network-data hops contribute 0.001001 s each, a completion-control message contributes 0.001 s, and Fpre and Fpost contribute 0.005 s each. Thus the additional latency is $3(0.001001) + 0.001 + 2(0.005) = 0.014003$ s. The two fixed services add $0.015 + 0.015 = 0.03$ cost units, while network costs are zero. The resulting limits are $\ell_G^{M0} = 0.014003 + \max_i \ell_i$, $c_G^{M0} = 0.03 + \sum_i c_i$ and $q_G^{M0} = \min_i q_i$. These constants come from the simulator configuration and must be recalculated for other graph configurations.

Applying these rules to the advertised I1 regions gives the M0-induced region A_G^{I1+M0}, which need not equal the requested graph-level SLA region A_G. For example, M0 may calculate an 80 ms latency limit while the requested SLA allows 100 ms. M0 makes a prediction only when $A_G^{I1+M0} \subseteq A_G$: its calculated maximum latency and cost do not exceed the requested maxima, and its minimum quality meets the requested minimum. Otherwise M0 returns **not applicable** instead of a numerical prediction. This status means M0 cannot estimate that prediction query; it does not mean the requested admissibility region is invalid or that the service will fail. Provided the required I1 values are available, the prediction is

$$\widehat{\sigma}_G^{M0}(A_G, H; \rho) = \prod_{i \in \{A,B,C\}} \widehat{\sigma}_i^{I1}(A_i(\rho), H; \rho). \tag{8}$$

M0 estimates the SLA probability from the providers' I1 cards; the white-box reference evaluates the requested graph-level region A_G directly. With fixed cards, H and ρ, M0 gives the same probability for every requested region containing A_G^{I1+M0}, even though the white-box probability can change with A_G. The estimate can also be wrong when the regions coincide: providers can fail on different requests, and the product in Eq. (8) does not account for those different failure sets or for execution coupling.

4.2 M1: one reconstructed provider process per provider

M0 estimates the end-to-end SLA-satisfaction probability (Eqs. (3) and (4)) using analytical composition rules and the probabilities advertised by individual providers. M1 follows a different approach. It reconstructs one stochastic execution model for each provider from its public I1 information, connects these models according to the known composition graph G, and estimates the end-to-end SLA probability through simulation. Unlike M0, M1 evaluates the requested graph-level SLA region A_G directly in the simulated executions of the reconstructed system.

Each reconstructed provider is represented by three parameters:

$$\theta_i = (\mu_i, \kappa_i, \mathrm{CV}_i), \tag{9}$$

where μ_i is the mean processing time, κ_i is the cost per unit of processing time, and CV_i is the coefficient of variation of processing time, defined as its standard deviation divided by its mean. Processing times follow a gamma distribution with the specified mean and variability. This parameterization defines the family of candidate provider models used by M1, M2 and M3.

For each provider, M1 searches for parameters whose simulated I1 SLA-satisfaction probabilities resemble those publicly advertised by that provider. Candidate models are compared with the published probabilities across the advertised regions, horizons and compliance thresholds using root mean squared error (RMSE). The search uses a Tree-structured Parzen Estimator (TPE) to propose candidate parameters. It has no

access to the provider's internal traces or generating parameters. Consequently, its objective is to construct a model consistent with the disclosed evidence, not necessarily to recover the provider's true implementation.

In the expanded multi-world evaluation (historical Phase-5 V3b), M1 performs seven independent TPE searches per provider, each evaluating 25 candidate configurations using 15 local simulation runs per candidate. The seven selected candidates are then reevaluated using a fresh common bank of 100 provider-local simulations. The candidate with the lowest I1 RMSE is retained for each provider. A second independent local bank is used only for diagnostics, not selection. Finally, the three selected provider models are combined and simulated within the native composition graph to estimate the requested end-to-end SLA probability. The historical Phase-3/4 implementation used a different search procedure, summarized in the method-version matrix.

4.3 M2: equal-weight finite model propagation

Like M1, M2 reconstructs stochastic provider models from public disclosures and simulates their composition using the known graph to estimate end-to-end SLA satisfaction. The difference is that M2 retains three reconstructed models per provider instead of one. It then simulates all $3^3 = 27$ combinations of the three providers' models and averages the resulting graph-level predictions with equal weights:

$$\widehat{\sigma}_G^{M2}(A_G, H; \rho) = \frac{1}{27} \sum_{j=1}^{27} \widehat{\sigma}_G(A_G, H; \rho \mid \vartheta_j). \tag{10}$$

Here ϑ_j denotes one joint model containing a reconstructed model for each provider, and the conditional prediction is obtained by simulating that model on graph G for the requested region A_G, horizon H and compliance threshold ρ. Equal weighting is a design choice, not an estimate of the probability that any reconstructed model is the true provider implementation.

In the expanded evaluation across multiple provider configurations and graphs (historical Phase-5 V3b), M2 uses the same seven I1-fitted candidates per provider described in Section 4.2 and retains the three with the lowest common-bank I1 RMSE. Each provider's M1-selected model is therefore included in M2's support, and the M1 joint model is one of the 27 combinations. This selection has no explicit diversity requirement or hard I1-compatibility threshold. The historical Phase-4 M2 instead selected diverse candidates and excluded the M1 anchor, as recorded in the method-version matrix.

Each of the 27 joint models is simulated for 100 graph trajectories, giving **2,700 graph trajectories per physical cell**. Shared seed indices across member simulations must be respected in numerical uncertainty estimates (Section 9.1). By retaining several provider reconstructions selected for their agreement with the public I1 evidence, M2 reduces the dependence of the graph-level prediction on a single fitted model. The hypothesis is that averaging predictions from these alternative reconstructions can mitigate errors specific to one model selection, especially when the disclosed evidence does not clearly favor a unique reconstruction.

Reconstruction with enriched provider information. M2 depends on the quality of the reconstructed provider models. Even when several models are retained, I1 discloses only probabilities of satisfying a finite set of compliance thresholds. We therefore investigated whether richer public evidence could better constrain provider reconstruction and improve subsequent graph-level prediction. I2a discloses the distribution of compliance across runs at each horizon, while I2b additionally discloses paired compliance fractions at two horizons within the same run (Section 3.2).

Six reconstruction variants were evaluated using the same M2 aggregation rule in Eq. (10). All use the same provider-model family and parameter domains, retain three models per provider, and propagate their 27 joint combinations with equal weights. The I1STRONG_M2 variant provides an I1-based reference under the same strengthened search budget. The variants differ in the **disclosed evidence used for fitting and the criterion used to select provider models**.

For each provider and variant, the matched search evaluates 256 parameter settings: 64 initial space-filling proposals followed by 192 adaptive proposals. Each candidate is scored with 25 provider-local simulation runs; the best 24 are rescored on a fresh common bank of 100 runs, and the best three are retained. The 27 resulting joint models are each simulated for 100 graph trajectories, giving **2,700 graph trajectories per variant and physical cell**. Search budgets and random-number banks are matched, although adaptive proposals can diverge because their fitting scores differ. Table 1 specifies the six disclosure-and-fitting variants.

Table 1: M2 disclosure-and-fitting variants used in the post-result P4/G_SEQPAR comparison. Each row identifies the provider evidence and fitting criterion; all six variants share the 27-member equal-weight graph ensemble and a 2,700-trajectory graph budget. Disclosure and fitting change together, so this is not an isolated test of information quantity.

M2 variant	Public evidence and reconstruction criterion
I1STRONG_M2	Fits the full I1 surface of published probabilities across advertised regions, horizons and compliance thresholds.
I2AFS_M2	Fits the I2a distributions of per-run compliance across all possible thresholds, rather than only the five thresholds published by I1.
I2B_M2	Fits the joint distribution of I2b compliance pairs at two horizons within a run, weighting the eight time separations equally and windows equally within each separation.
I2BVAR_M2	Uses the same I2b pair-matching loss but gives more weight to windows with larger average squared changes between endpoints; weights are normalized globally across windows.
I2BHIER_M2	Uses the same I2b pairs, weighting larger endpoint changes more heavily within each region and separation, while preserving equal total weight for the five regions and eight separations.
I2BV4_M2	Uses the I2b pair-matching loss and the additional public mean within-interval range (Section 3.2). The range determines fitting weights; it is not itself an additional direct fitting target.

These variants differ not only in what providers disclose (I1, I2a or I2b), but also in how the evidence is used to fit reconstructed models. Their comparison therefore evaluates complete disclosure-and-fitting procedures, not the isolated contribution of additional information. The tests were conducted after earlier results were known, on the previously studied P4/G_SEQPAR cell, and are **post-result development evidence**, not an independent confirmation (historical Phase-6; see Section 8.3).

4.4 M3: public-evidence-weighted finite reconstruction

M1 selects one reconstructed model per provider, and M2 averages predictions from three equally weighted models per provider. M3 instead retains a larger set of candidate reconstructions and gives more influence to those that better reproduce the provider's public I1 information. The motivation is to retain alternative explanations of the disclosed evidence without assuming they should all contribute equally.

In the expanded evaluation, **M3 uses the same seven provider candidates generated by the reconstruction procedure in Section 4.2**. No additional provider-parameter search is performed for M3. Whereas M1 selects the lowest-I1-RMSE candidate and M2 retains the three lowest-I1-RMSE candidates, M3 retains all seven and computes a separate evidence-based weight for each. All use the same gamma-process family and parameterization in Eq. (9).

To measure agreement with I1, let $\mathcal{R}_i$ be the advertised region, compliance-threshold and horizon combinations with $H > 0$ for provider i. At observation $r \in \mathcal{R}_i$, let $p_{i,r}$ be the published probability of satisfying the SLA requirement and $q_{i,k,r}$ the corresponding probability simulated from candidate k. M3 uses the mean Bernoulli Kullback–Leibler (KL) discrepancy

$$\begin{aligned} E_{i,k} &= \frac{1}{|\mathcal{R}_i|} \sum_{r \in \mathcal{R}_i} d_{\mathrm{B}}(\widetilde{p}_{i,r}, \widetilde{q}_{i,k,r}), \\ d_{\mathrm{B}}(p, q) &= p \log \frac{p}{q} + (1-p) \log \frac{1-p}{1-q}. \end{aligned} \tag{11}$$

The tildes indicate Jeffreys smoothing: for an empirical probability p estimated from N runs, $\widetilde{p} = (Np + 1/2)/(N+1)$, and likewise for q using its own number of runs. In the expanded evaluation both the published I1 estimates and the common-bank candidate estimates use $N = 100$. Smoothing avoids zero or one probabilities producing undefined or infinite logarithmic discrepancies.

Why KL rather than RMSE? RMSE measures absolute discrepancies between advertised and reconstructed compliance probabilities, regardless of their values. However, the same absolute error can have different implications for reliability. For example, predicting 0.99 when the advertised compliance probability is 0.90 underestimates the probability of noncompliance by a factor of ten, whereas a comparable error near 0.50 has a much smaller relative effect. Bernoulli KL penalizes such discrepancies more strongly because it measures the excess expected logarithmic loss of a candidate's probability predictions. We use this property to give less weight to reconstructed models that make overconfident predictions inconsistent with the public

evidence. Whether this weighting improves graph-level prediction compared with RMSE-based weighting remains an empirical question not isolated by the present experiments.

Candidate k for provider i receives the normalized weight

$$w_{i,k} = \frac{\exp[-\lambda E_{i,k}]}{\sum_{k'} \exp[-\lambda E_{i,k'}]}, \tag{12}$$

where $\lambda \geq 0$ determines the concentration of weights: $\lambda = 0$ gives equal weights, while larger values favor smaller KL discrepancies. In the original pilot, $\lambda = 30$ was selected by five-fold cross-validation on the public I1 probability surface, without access to the graph white-box reference. The expanded evaluation **reused the value 30 without repeating that cross-validation**; Section 8.2 reports the consequences of that protocol difference. These Gibbs weights express relative support within a finite candidate bank; they are **not calibrated posterior probabilities** that candidates are the hidden generating models.

For a joint reconstruction $\vartheta_j = (\theta_{A,a}, \theta_{B,b}, \theta_{C,c})$, M3 multiplies the three provider weights:

$$W_{a,b,c} = w_{A,a} w_{B,b} w_{C,c}. \tag{13}$$

This factorization concerns weights assigned to candidate provider models. It does not assume independent graph execution: queueing, request histories and branch dependencies remain coupled within the native simulation of each joint model. For a propagated support S, with normalized weights $\alpha_j = W_j / \sum_{k \in S} W_k$, M3 predicts

$$\widehat{\sigma}_G^{M3}(A_G, H; \rho) = \sum_{j \in S} \alpha_j \widehat{\sigma}_G(A_G, H; \rho \mid \vartheta_j). \tag{14}$$

The **initial controlled pilot (historical M3-v4)** used a different candidate-generation procedure: 48 candidates per provider generated by *Latin hypercube sampling* (LHS), rather than seven independently fitted candidates. LHS spreads candidate parameter combinations across predefined ranges by dividing each parameter range into 48 strata and using one sampled value from each stratum per parameter. Unlike an adaptive search, these proposals are not updated in response to how well earlier candidates fit I1. This provides space-filling coverage of the specified ranges, not a guarantee of finding a sufficiently accurate reconstruction. With public-I1 Gibbs weights at $\lambda = 30$, the pilot propagated the **Top14** joint models using graph budgets of $B = 1400$ or 2000. Those 14 members held about 99.917% of the weight **within that finite inferred distribution**, not of the probability of the true provider system.

The subsequent expanded evaluation (historical Phase-5 V3b) used the seven candidates per provider described above and propagated **all** $7^3 = 343$ **joint models** with total graph budget $B = 1400$. The initially registered Top14 approximation was replaced before the final graph white-box results were read because a permitted pre-graph diagnostic found that Top14 retained only 0.552 of the inferred mass in P1, 0.962 in P3 and 0.867 in P4. This preserved the candidate bank and Gibbs weights, but changed numerical propagation and therefore remains a **documented method revision**, not an unchanged test of the original Top14 method.

For any retained subset of total Gibbs weight m_S, omitting the remaining reconstructed models changes the exact weighted-portfolio prediction by at most $1 - m_S$. The maximum difference was approximately 0.000826 for the original pilot Top14 and zero for FULL343 within its own 343-member bank; neither value bounds prediction error relative to the real composed service.

Allocation of the graph-simulation budget. After weighting the reconstructed joint models, M3 still needs graph simulations to estimate their individual SLA probabilities. In the expanded evaluation, all 343 joint models share a budget of $B = 1400$ trajectories. Allocating the same number of runs to every model would ignore their unequal influence on the weighted prediction. Instead, M3 allocates runs to reduce an upper bound on the prediction's Monte Carlo variance.

For a propagated support S with normalized weights α_j (Eq. (14)), let N_j be the number of independent graph trajectories simulated for model j. Each trajectory records whether the requested compliance target is met. Assuming independent simulation estimates across joint models, the conditional numerical variance satisfies

$$\mathrm{Var}_{MC}(\widehat{\sigma}_G^{M3}) \leq \frac{1}{4} \sum_{j \in S} \frac{\alpha_j^2}{N_j}, \qquad \sum_{j \in S} N_j = B. \tag{15}$$

For fixed models, weights and simulation counts, the bound uses the elementary fact that a Bernoulli indicator with success probability p_j has variance $p_j(1-p_j) \leq 1/4$. Minimizing this **worst-case numerical variance bound** with a continuous trajectory budget gives $N_j = B\alpha_j$, a special case of the weighted allocation principle of Neyman (1934; Appendix D). This is not an allocation optimized for the unknown member-specific variances.

For integer counts $N_j \geq 1$, adding one trajectory to member j reduces the bound by $\alpha_j^2/[4N_j(N_j+1)]$. M3 starts with one trajectory per joint model and assigns each remaining trajectory to the largest reduction, with deterministic rank-based ties. This is a marginal-priority integer allocation of the kind studied by Wright (2014; Appendix D). The factor 1/4 is common to all members and can be omitted when ranking priorities. In the executed FULL343 support, $\alpha_j = W_j$, giving the implemented rule $\Delta_j = W_j^2/[N_j(N_j+1)]$.

A separate read-only verification script `verify_method_support_v0_1.py` reproduced the archived 343-member, $B = 1400$ allocation counts for P1, P3 and P4; its findings are recorded in `METHOD_SUPPORT_VERIFICATION_REVIEW_2026-10-16.md` (Appendix B.1). This checks agreement between the saved allocations and the specified rule. It does **not** demonstrate improved prediction accuracy, verify every simulation trajectory or address error caused by the reconstructed provider models.

M3 Top1 is not necessarily M1. M1 selects by minimum I1 RMSE, while M3's highest-weight joint model is determined by the product of KL-based provider weights. More generally, comparing M3 with M2 also changes candidate coverage, weighting loss and graph-simulation allocation. An observed difference in prediction accuracy therefore does not isolate the effect of KL weighting alone.

5. Native simulator, graph structures, and experiment protocol

5.1 AICon-based simulation framework and composition extensions

The experiments use AICon (Agentic Intelligence for the Computing Continuum) [11], built on a modified fork of YAFS (Yet Another Fog Simulator) [9]. YAFS provides a Python-based discrete-event environment for simulating applications across edge, fog and cloud infrastructure, including network communication, service execution and queueing. AICon adds a management-agent layer through which agents can observe simulated system conditions and intervene in their operation.

Why use AICon rather than extend YAFS directly? The choice connects this performance-prediction testbed to the broader research programme on Composable Autonomic Offerings (CAO) [12]. In future work, management agents could use runtime observations and performance predictions to investigate **local resilience**: detecting degradation and considering changes to service offerings or compositions before operational objectives are lost. This is a direction for subsequent CAO experiments, **not a capability validated by the present results**. In particular, the prediction accuracy demonstrated here is not assumed sufficient for dependable autonomous intervention.

For the present study, the modified AICon/YAFS runtime provides service-composition mechanisms beyond individual service execution. It registers branches and their execution dependencies, tracks the progress of each composition request separately, releases a dependent branch only after the required predecessor results for that request have arrived, and emits a single result when all required branches complete. Standalone engineering probes exercise joins, isolation between concurrent composition requests and nested composition behavior. These mechanisms let us measure end-to-end service outcomes under sequential and parallel dependencies, including the effects of queueing and network delays.

A second extension introduces **stochastic instruction demand** for simulated application modules. Instead of assigning the same instruction count to every invocation, a message can carry a distribution from which the instruction count is sampled when that message is instantiated. The simulator converts the sampled instruction demand into execution time using the processing capacity of the infrastructure node executing the application module, expressed in instructions per simulation time unit (IPT). In these experiments, provider work is Gamma-distributed, with a deterministic case when the specified variation is zero [23]. Thus, successive requests can take different times on the same infrastructure, with the resulting variation interacting with queueing and composition dependencies.

The expanded Phase-5 evaluation (Phase II) tests the prediction methods beyond the initial single-world, single-graph pilot, across several service-composition graphs G and provider worlds. Here G denotes the **logical dependencies among provider services**. The battery considers four fixed three-provider graphs: fully parallel, fully sequential, and two mixed sequence/parallel structures. For each provider world, the

four compositions use the same physical infrastructure and provider placements. Only the execution dependencies among the providers change. Periodic arrivals trigger stochastic provider services, with native queueing, service cost and network delays. The experiment covers these four structures, rather than arbitrary nested service graphs; Section 5.2 describes the provider-world and SLA-query variations.

Table 2 defines the four frozen dependency maps and their logical forms.

Table 2: Four frozen service-composition graphs used in the experiments. The predecessor columns give immediate dependencies for providers A, B and C; "none" denotes no preceding provider. Logical forms use $\|$ for parallel-all and $\rightarrow$ for sequence. Physical infrastructure and provider placement are held fixed across these graphs.

Logical graph	A predecessors	B predecessors	C predecessors	Reading
G_PAR	none	none	none	$A \parallel B \parallel C$
G_SEQ	none	A	B	$A \rightarrow B \rightarrow C$
G_SEQPAR	none	A	A	$A \rightarrow (B \parallel C)$
G_PARSEQ	none	none	B	$A \parallel (B \rightarrow C)$

The simulator can also represent device power consumption and the quality of a service's output. In this evaluation, the power-consumption parameter (WATT) is set to zero for every simulated device. AICon can vary output quality according to the fraction of an application's required instructions that are executed, but here each module executes its full instruction demand, so this mechanism does not introduce quality variation. Although the SLA objective includes latency, cost and quality (L, C, Q), the experiments primarily investigate latency and cost, not changes in energy consumption or output quality.

5.2 Experimental design

To evaluate the prediction methods under different conditions, we considered four provider configurations (P1–P4) and the four service-composition graphs introduced in Section 5.1, giving 16 planned test cases. For Step-0, a *query condition* (A, ρ) fixes the admissibility region and required compliance fraction, but not the horizon; evaluating it at a particular H gives a complete prediction query $(A, H; \rho)$ (Section 2.2). Step-0 classified these conditions by their estimated probability of meeting the compliance requirement: higher probabilities meant easier conditions, and lower probabilities harder. For each condition, this probability was estimated using preliminary white-box simulations and averaged over the 37 horizons $H = 60, 65, \ldots, 240$ seconds. For each of five fixed compliance thresholds ρ, it sought one Easy condition (mean probability in $[0.95, 1.00]$), one Mid condition ($[0.75, 0.90]$), and one Stress condition ($[0.40, 0.75]$). Adjacent regime means had to differ by at least 0.10. Candidate conditions were obtained by scaling the latency and cost limits of the publicly composed baseline SLA region by $s = 0.50, 0.51, \ldots, 2.00$, keeping the quality limit fixed, and requiring $s_{\text{Stress}} < s_{\text{Mid}} < s_{\text{Easy}}$.

For each test case, a method-independent calibration bank of 200 white-box trajectories selected candidates closest to the respective regime target centers. A separate, disjoint bank of 200 trajectories then had to confirm the same probability bands and minimum separations for all 15 query conditions. If any required condition could not be selected or confirmed, the whole world-by-graph case was excluded without reselection or replacement. All four P2 cases failed Step-0, leaving 12 cases for evaluation. These Step-0 trajectories served query design only; they were not used to fit the prediction methods or as the final white-box reference.

For each retained case, we evaluated 15 query conditions at 37 observation horizons, from 60 to 240 seconds in steps of five seconds. This gives 555 probability estimates per case and 6,660 across the 12 cases. These measurements are related because different SLA conditions and horizons can be evaluated using the same simulation trajectories.

The prediction methods received the published provider information and the composition graph, but not the underlying provider models. Their predictions were recorded before being compared with a separate reference generated from 1,000 new simulations of each composition using its actual provider models. This reference, which we call *white-box*, provides an independent estimate of the SLA satisfaction probability against which prediction accuracy can be measured.

5.3 Evaluation safeguards and reproducibility

As summarized in Appendix C, the research progressed from a controlled pilot (Phase I) to a broader evaluation (Phase II) and subsequent retrospective investigations (Phase III). Here we focus on how the expanded evaluation separated predictions from their reference results, what the recorded evidence verifies, and which reproducibility limitations remain.

To evaluate prediction accuracy, the predictions must be recorded before the reference results used to assess them become available. For each of the 12 eligible test cases, the methods first estimated end-to-end SLA satisfaction using the information published by the providers. These predictions were then fixed, before a separate set of simulations using the underlying provider models generated the reference probabilities. This separation allowed the predictions to be evaluated without using the final reference results to construct them.

A subsequent audit examined whether this separation was supported by the saved experimental records. For all 12 cases, the saved prediction files matched the hashes recorded when predictions were frozen, and the final-reference records linked back to that same frozen prediction set.

The methods and procedures also changed during development. Provider-model reconstruction and the integration of reconstructed models were revised in response to earlier diagnostics, before the final reference results were examined. Some engineering checks were completed after scientific experiments had already begun. Appendix C identifies these revisions and their scientific roles, so the expanded evaluation should not be interpreted as an unchanged test of the original plan. The repository retains source versions, simulation configurations and verification records, but the exact AICon/YAFS software state used in every historical run has not been fully established. Reference [23] identifies the audited AICon checkout; complete bitwise reproduction of the original runs is therefore not claimed.

6. Controlled pilot (Phase I)

The controlled pilot (Phase I) provided the first evaluation of the prediction methods using a single provider configuration and composition graph. It established an initial comparison of their prediction accuracy under fixed experimental conditions. The results also motivated the broader evaluation presented in the following section. As detailed in Appendix C, the pilot and subsequent evaluations involved different method versions and therefore serve distinct scientific purposes.

6.1 Construction of the pilot reference

The pilot evaluated one fixed service composed of three providers operating in parallel, with a new request generated every 0.2 seconds. To measure prediction accuracy, we needed reference probabilities for the actual composed service. We therefore simulated it repeatedly using the underlying provider models, which were not available to the prediction methods. Each simulation produced request outcomes from which we could assess end-to-end SLA compliance at different observation horizons. The provider models and their published I1 information remained fixed throughout the pilot.

The simulation outcomes could be assessed against many different SLA requirements, so we first had to choose which requirements to test. In the original pilot, we used the advertised provider limits as a starting point. We combined them using M0's aggregation rules (Section 4.1): for this parallel graph, the baseline latency is the largest provider latency plus fixed delays, the costs are added together with fixed costs, and the baseline quality is the lowest provider quality. This defines the baseline SLA region $A_G^{\text{base}}(\rho) = A_G^{I1+M0}(\rho)$. We then scaled its latency and cost limits by a factor γ, leaving the quality limit unchanged:

$$A_G(\gamma, \rho) = \{L_G \le \gamma \ell_G^{\text{base}}(\rho),\ C_G \le \gamma c_G^{\text{base}}(\rho),\ Q_G \ge q_G^{\text{base}}(\rho)\}. \tag{16}$$

The scaling factor specifies the difficulty of the requested SLA: $\gamma > 1$ allows more latency and cost, whereas $\gamma < 1$ imposes tighter limits. We combined three such difficulty levels with five compliance thresholds ρ, giving 15 SLA conditions. Each condition was evaluated at 37 observation horizons, from 60 to 240 seconds, using the same bank of $N_{\text{WB}} = 200$ independent simulation trajectories. For each trajectory and horizon, we calculated the fraction of decided requests satisfying the requested SLA region. The reference probability

was the proportion of trajectories in which that fraction reached the required threshold ρ. Because the same trajectories were evaluated repeatedly, these reference measurements are related.

Using M0's aggregation rules to choose the SLA test conditions is separate from using M0 to predict SLA satisfaction. The reference probabilities were obtained from simulations of the actual composed service, independently of M0's predictions.

6.2 Prediction errors and historical cutoff diagnostic

Table 3 compares the prediction errors of the four methods against the pilot's simulation-based reference probabilities:

Table 3: Phase I pilot graph-level prediction errors against a fresh white-box reference ($N_{\mathrm{WB}} = 200$), on one fixed parallel graph and provider world. The pilot comprises 15 query conditions at 37 horizons (555 related points); M0 returns numerical estimates for only 333 of the 555 prediction queries. Graph trajectories count composed-service simulation runs per method. MAE is mean absolute error; RMSE is root mean squared error; positive signed bias denotes overprediction. M0 RMSE is not reported.

Method, pilot	Graph trajectories	Scored queries	MAE	RMSE	Signed bias
M0	0	333	0.57204	Not reported	-0.57204
M1	100	555	0.18517	0.23074	+0.18517
M2, 27 equal members	2,700	555	0.06815	0.09414	+0.04737
M3 pilot Top14, $B = 1400$	1,400	555	**0.03037**	0.03578	+0.02824
M3 pilot Top14, $B = 2000$	2,000	555	**0.02755**	0.03306	+0.02447

MAE, RMSE and bias are computed against the WB reference. Positive bias means optimistic prediction. M0 returned numerical predictions for 9 of the 15 query conditions (333 of the 555 complete prediction queries). For the other six query conditions, M0 returned not applicable because the required latency and cost limits were tighter than those it calculated from the providers' advertised information. This does not imply that the service would fail those conditions. Its aggregate error is based on fewer prediction queries than those of the other methods, which were evaluated on all 555 queries. M3 used 1,400 simulations of the composed service, compared with 2,700 for M2. However, these counts exclude the computational cost of reconstructing the provider models, so they do not establish which method is less expensive overall.

Besides measuring probability errors, the pilot also examined whether the methods made the same yes/no decision as the WB reference. The quantity being predicted, defined in Eq. (4), is the probability that at least a fraction ρ of decided requests meets the SLA. For this additional analysis, we used an illustrative probability threshold $\beta = 0.90$:

$$\sigma_G(A, H; \rho) = \Pr\{c_G(A, H) \geq \rho\},$$

$$d_M(A, H; \rho, \beta) = \begin{cases} 1, & \widehat{\sigma}_G^M(A, H; \rho) \geq \beta, \\ 0, & \widehat{\sigma}_G^M(A, H; \rho) < \beta. \end{cases}$$

Here ρ is the required fraction of compliant requests, while β is the minimum predicted probability of achieving that fraction. A decision of 1 means the method predicts a probability of at least 0.90; otherwise the decision is 0. We obtained the corresponding WB decision by applying the same threshold to the WB reference probability. The choice $\beta = 0.90$ was made for this secondary comparison and was not part of the contractual SLA requirements.

Because the WB reference is estimated from simulations, its probabilities contain statistical uncertainty. When an estimate is close to the 0.90 threshold, its yes/no classification may be uncertain. We therefore retained only measurements whose uncertainty interval lay entirely above or below the threshold (technically, 95% Wilson confidence intervals). This left **481 of the 555 measurements** for comparing decisions. Of these, **274** were measurements for which M0 could produce a prediction. The observed agreement with the WB decisions was **33.94%** for M0 (on 274 measurements), **37.63%** for M1, **86.07%** for M2, **96.47%** for M3 with 1,400 simulations, and **97.09%** for M3 with 2,000 simulations (the latter four on 481 measurements).

The pilot's M3 production errors were substantially smaller than those of M1/M2 in this exact world and graph. That is a genuine, archived numerical observation. It does not demonstrate the same numerical advantage in worlds with altered provider demand or different logical dependencies.

6.3 Retrospective support and weighting

To understand the role of model selection, weighting, and simulation allocation in the pilot, we compared alternative predictors on the same provider configuration and graph. Table 4 presents their prediction errors, with each method using a total of 1,400 simulations of the composed service.

Table 4: Retrospective Phase I pilot estimator comparison, with 1,400 composed-service simulations per variant and errors measured against the same pilot white-box reference. Top1 uses the highest-weight joint model. Uniform14 and Weighted14 with equal allocation simulate each of 14 selected models 100 times and differ only in averaging weights; production-allocation Weighted14 changes the allocation and random simulation seeds while retaining the same models and weights. MAE, RMSE and bias are probability-scale errors; positive bias denotes overprediction.

Retrospective pilot estimator	MAE	RMSE	Bias
Top1 B1400	**0.02878**	0.03406	+0.02498
Uniform14 equal-allocation B1400	0.05490	0.06570	+0.05362
Weighted14 equal-allocation B1400	0.04222	0.05016	+0.04066
Weighted14 production-allocation B1400	0.03037	0.03578	+0.02824

The two equal-allocation variants use the same simulated trajectories, so their difference shows the effect of changing the averaging weights. In contrast, production allocation changes both the number of simulations assigned to each model and the random simulation seeds. Its lower error therefore cannot be attributed to allocation alone. Top1 also achieved a slightly lower MAE than production Weighted14 in this pilot, showing that averaging several models was not necessary for the lowest observed error under these conditions.

6.4 Hidden-parameter recovery

A retrospective analysis compared the actual provider parameters used in the simulations with those reconstructed by M1 and M3-Top1. In this pilot, some reconstructed parameters, particularly for Provider C, differed substantially from the actual values, even when graph-level prediction errors were small. This shows that accurate graph-level prediction does not necessarily require accurate recovery of the hidden provider parameters.

7. Expanded evaluation (Phase II)

7.1 Eligibility and exact prediction population

Phase-5 planned 16 physical world-by-graph cells. As described in Section 5.2, Step-0 used preliminary simulations of the underlying provider models to select informative SLA conditions and confirm each cell's eligibility. All four P2 cells failed this check and were excluded without replacement. The remaining **12** cells cover worlds P1, P3, P4 across G_PAR, G_SEQ, G_SEQPAR and G_PARSEQ. Each has 15 frozen query conditions, each evaluated at 37 horizons; each has a distinct fresh $N_{\mathrm{WB}} = 1000$ graph reference.

Phase-5 also planned to compare how prediction error changes between parallel and sequential compositions in symmetric provider worlds (P1/P3) and unequal-load worlds (P2/P4). Because all four P2 cases failed Step-0, this comparison could not be completed as planned. A later experiment involving P2 used different SLA conditions and cannot replace the missing results.

7.2 Executed V3b method comparison

The Phase-5 evaluation used V3b-selected candidate banks: **M1 = one joint model, M2 = 27 equal-weight joint members, M3 = FULL343 weighted joint members**. Table 5 reports the independently checked aggregate prediction errors on the eligible Phase II population:

Table 5: Phase II aggregate prediction errors for executed Phase-5 V3b methods on 12 Step-0-eligible provider-world-by-graph cells (P1, P3 and P4 across four graphs). Each of M1–M3 is scored on 6,660 related prediction queries, using a separate fresh white-box reference with $N_{\mathrm{WB}} = 1000$ per cell; M0 returns numerical predictions for only 1,776 of those queries. Graph simulation budgets differ (M1: 100, M2: 2,700, M3: 1,400 per cell). MAE, RMSE and signed bias are relative to the white-box reference; positive bias indicates overprediction. These are not cost-matched methods.

Phase-5 executed V3b	Scored queries	Graph trajectories/cell	MAE	RMSE	Signed bias
M0	1,776	0	0.41874	0.45392	-0.41808
M1	6,660	100	0.15702	0.22588	+0.14968
M2	6,660	2,700	0.13395	0.19045	+0.10094
M3 FULL343	**6,660**	**1,400**	**0.12189**	**0.18081**	**+0.06511**

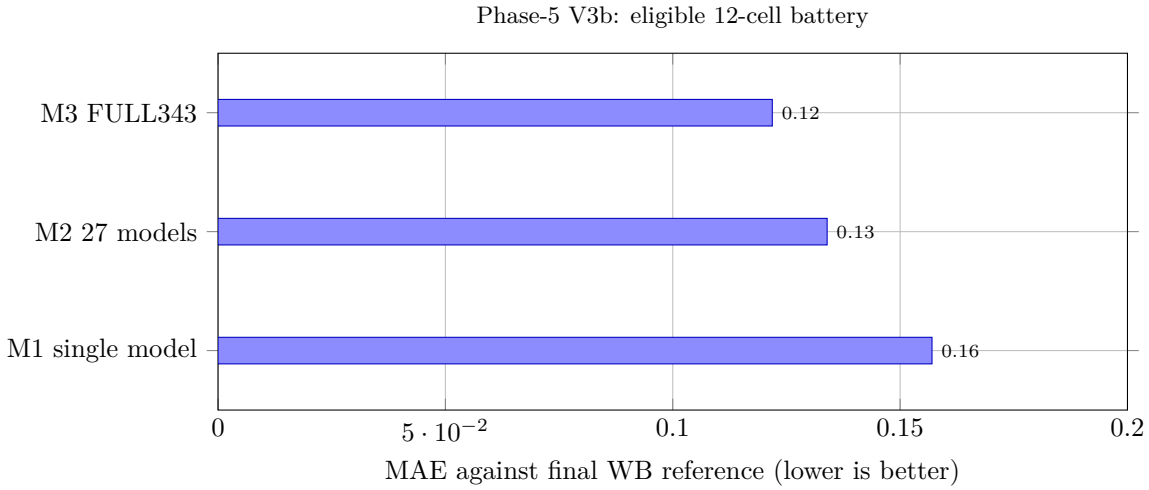


Figure 1: Phase-5 aggregate MAE from verified frozen V3b pointwise results. The full 12-cell eligible battery contains 6,660 related SLA measurements (across conditions and time horizons) per model. Different graph budgets apply: M1 100, M2 2,700 and M3 1,400 trajectories per cell. M0 is omitted because it returns numerical predictions for only 1,776 of the 6,660 prediction queries.

Table 5 reports errors in the estimated end-to-end SLA satisfaction probability. M3 FULL343 has the lowest aggregate MAE among M1–M3, at 0.12189 over 6,660 related query-horizon points. This corresponds to an average absolute difference of about 12.2 percentage points between M3's predictions and the finite white-box reference. M1, M2 and M3 have positive aggregate signed biases, indicating that they overpredicted SLA satisfaction on average.

The expanded evaluation covers 12 eligible cases across three provider configurations and four composition graphs, providing broader evidence than the single-graph pilot. However, an average error over the full evaluation does not show how prediction accuracy varies across composition graphs, SLA requirements, or observation horizons. This matters for management decisions: an inaccurate estimate near an admission threshold may lead to accepting a service that should have been rejected, or vice versa. The result therefore motivates further investigation of how to select evaluation scenarios, determine simulation budgets, and measure errors and uncertainties at the level of individual decisions, rather than relying only on aggregate accuracy.

M0 produced estimates for only 1,776 of the 6,660 prediction queries, whereas M1, M2 and M3 were evaluated on all 6,660. Their aggregate errors therefore refer to different sets of queries and cannot be compared directly. Moreover M1/M2/M3 have sharply different graph simulation budgets, so the aggregate

ranking does not isolate simulation efficiency at equal cost, nor does the pre-WB change from Top14 to FULL343 permit a direct unchanged-algorithm replication of the September pilot.

7.3 Topology and provider-world effects

Why compare composition graphs? The pilot evaluated one parallel composition. For an individual request, a parallel join finishes when its slowest required branch finishes. An inaccuracy in estimating the execution time of a faster branch need not change the predicted join latency if that branch remains faster than the slowest branch. In a sequence, the latencies of all required providers add, so inaccuracies in their estimated execution times can influence the total. This reasoning concerns latency; whether it leads to higher error in the predicted graph-level SLA satisfaction probability, which also depends on cost and quality, must be tested. Before examining the expanded-evaluation results, we therefore specified a hypothesis: *for the same provider world and prediction method, would the sequential graph produce larger prediction errors than the parallel graph?* We recorded the comparison as the difference between their mean absolute errors (MAE):

$$\Delta_P^M = \mathrm{MAE}_M(P, G_{\mathrm{SEQ}}) - \mathrm{MAE}_M(P, G_{\mathrm{PAR}}). \tag{17}$$

Here $\mathrm{MAE}_M(P, G)$ is the average absolute difference between the SLA-satisfaction probabilities predicted by method M and the white-box reference for provider world P and composition graph G. A **positive** Δ_P^M means that the sequential graph had higher average prediction error in the evaluated queries; a **negative** value means that the parallel graph had higher error. The original hypothesis anticipated a positive difference.

The results did not show a consistent direction. For M3 FULL343, Δ_P^M was -0.0448 in P1, $+0.0563$ in P3 and $+0.0196$ in P4. Sequential composition therefore had lower observed MAE in P1 but higher observed MAE in P3 and P4. These differences do not establish that sequence is generally harder to predict. In addition, Step-0 selected SLA query conditions separately for each world-by-graph case, so this comparison does not isolate the effect of graph structure alone.

A second planned comparison would have contrasted the topology differences between the more symmetric provider worlds (P1/P3) and the unequal-load worlds (P2/P4). It could **not be completed** because all P2 cases were excluded by Step-0. The two mixed graphs, G_SEQPAR and G_PARSEQ, provide additional observations, but they also do not separate the effect of graph structure from changes in workload reaching each provider.

The expanded evaluation tested 12 provider-world-by-graph configurations. Each was simulated using $N_{\mathrm{WB}} = 1000$ separate final white-box trajectories. Within each configuration, those same trajectories were evaluated against 15 SLA conditions at 37 horizons. Thus the 6,660 resulting probability measurements cover many SLA queries, but only 12 distinct tested configurations. This distinction matters when interpreting how broadly the observed topology effects can be generalized.

Intermediate methodological conclusion. The expanded evaluation provides broader evidence than the original single-graph pilot, but also reveals important challenges for empirical validation. Prediction accuracy varied across provider configurations and composition graphs, while the predefined SLA-condition selection procedure excluded four planned cases. Using more simulation trajectories can improve the precision of reference estimates, but it does not establish the reliability of reconstructed models across configurations. Moreover, evaluating many SLA queries on shared simulation trajectories increased coverage of operating conditions without creating an equivalent number of independent experimental settings.

These observations highlight several distinct dimensions of empirical validation: the choice and diversity of test configurations, the selection of operating conditions, the number of independent simulation runs, and the measurement of prediction errors and uncertainties. The present results provide initial empirical motivation for investigating these dimensions systematically. Developing and validating such a methodology for management and control algorithms in hierarchical computing-continuum systems is the objective of subsequent research.

8. Retrospective investigation (Phase III)

In the pilot (Section 6), the original M3 estimator achieved a mean absolute error (MAE) of 0.03037 on one parallel graph. In the expanded Phase II evaluation (Section 7), the revised M3 FULL343 estimator had an

aggregate MAE of 0.12189 across 12 provider-world-by-graph cases, with prediction errors varying across those cases. These results are not a direct replication: both the evaluated configurations and aspects of the reconstruction procedure changed. Phase III therefore examines which changes are documented, which numerical behaviors were preserved, and which explanations for the different results remain untested.

The investigation has three parts. Section 8.1 applies key rules of the original M3 method to the eligible Phase II cases and compares the results with the V3b implementation evaluated in Section 7. Section 8.2 reports ten named pilot-to-battery audit checks (D0–D9), covering simulator consistency, reconstructed candidate models, weighting, and parameter-selection rules. Sections 8.3–8.5 report additional diagnostic tests on provider disclosures, an alternative P2 evaluation, and reconstruction search. These investigations were carried out after the earlier results were known. Some reuse the Phase II white-box reference; others use separate diagnostic conditions. They help identify discrepancies and formulate hypotheses, but do not provide a new independent test of performance on unexamined systems.

8.1 Original-style M3 transfer to the Phase II battery

Why revisit the original M3 method? The pilot in Section 6 found a mean absolute prediction error (MAE) of 0.03037 for the original weighted Top14 estimator on one parallel composition. Phase II subsequently evaluated a revised M3 implementation, V3b FULL343, on a broader battery (Section 7). Since the candidate-generation and model-propagation procedures had changed, the Phase II results did not show how the *original-style* M3 procedure would have performed on that battery. We therefore reconstructed its Top14 estimator, alongside a single-model Top1 comparison, and evaluated both retrospectively on the existing Phase II cases. This investigation was conducted after the Phase II results had been examined.

How was the comparison made? All three battery estimators were evaluated on the same **12 eligible provider-world-by-graph configurations** as Phase II: P1, P3 and P4, each with G_PAR, G_SEQ, G_PARSEQ and G_SEQPAR. For each configuration, it reuses the **15 SLA conditions** selected in Step-0 (five compliance thresholds, each with Easy, Mid and Stress conditions) and the **37 observation horizons** from 60 to 240 seconds. That gives 555 prediction queries per configuration and 6,660 across the battery. The reference probabilities are those already obtained from 1,000 white-box trajectories per configuration. Section 5.2 explains the condition-selection procedure; the four P2 configurations were excluded in Phase II and were not reinstated.

What was reconstructed? For each provider, the original-style method generated **48 candidate stochastic models** by Latin hypercube sampling (LHS), a nonadaptive way to spread proposals across the parameter ranges. Each candidate was simulated for **100 provider-local trajectories** to estimate its I1 SLA-probability curve, which was compared with the provider's publicly disclosed probabilities. The public I1 information, provider-model family and M3 weighting rules are described in Section 3.1, Section 4.2 and Section 4.4, respectively. The resulting smoothed KL discrepancies were converted into Gibbs weights ($\lambda = 30$); multiplying provider weights gave $48^3 = 110{,}592$ possible joint models. For each configuration, the **14 highest-weight models (Top14)** shared a budget of $B = 1400$ graph trajectories, allocated according to the original variance-bound rule. The highest-weight model alone (**Top1**) was separately evaluated with the same budget. The resulting predictions were recorded before the retrospective evaluation read the saved white-box reference. The Phase II FULL343 predictions were kept unchanged. This is a reconstruction of the original-style rules, not an exact replay of the pilot.

Results by evaluation stage. Table 6 presents the pilot and expanded-battery results as *separate experiments*, identifying when each estimate was obtained. In particular, the last two rows are the **Phase III retrospective evaluation** of the original-style Top14 and Top1 methods on the Phase II test cases.

Table 6: Prediction errors shown in chronological experimental groups. The pilot results use one parallel composition with 555 related prediction queries (Top1 was added in a later analysis of that pilot; Section 6.3). The Phase II and retrospective Phase III rows use the same 12 eligible provider-world-by-graph configurations, with 6,660 related queries per method and the same Phase II white-box reference. All estimators used B=1400 graph trajectories per configuration. Signed bias above zero indicates overprediction. The pilot and expanded battery have different test conditions, so their errors are not directly matched.

Evaluation stage	Estimator	Tested scope (queries)	MAE	RMSE	Signed bias
Pilot (Phase I)	Original weighted Top14	1 graph (555)	0.030370	0.035780	+0.028240
Pilot follow-up	Original Top1	1 graph (555)	0.028780	0.034060	+0.024980
Expanded (Phase II)	V3b FULL343	12 cases (6,660)	**0.121890**	**0.180806**	+0.065105
Phase III	Original-style Top14	12 cases (6,660)	0.154155	0.212843	+0.044867
Phase III	Original-style Top1	12 cases (6,660)	0.162280	0.225373	+0.054562

Why did error rise if $B = 1400$ was unchanged? That number counts only graph simulations. The pilot tested one parallel service with SLA limits derived from provider disclosures; the battery tested 12 provider-world-by-graph cases with SLA limits selected separately for each case (Sections 6.1 and 5.2). Their white-box references also used different sample sizes (200 versus 1,000 trajectories). The predictor was not identical either: the same sampling positions were mapped to new parameter ranges, $\lambda = 30$ was reused without repeating the pilot's cross-validation, and Top14 was fixed rather than reselected by the pilot's 99.9%-weight rule (Section 8.2). Equal graph budgets therefore do not mean equal test conditions or identical reconstructed models.

What do the results establish? Original-style Top14 MAE rose from 0.03037 in the pilot to 0.15416 on the battery (Top1: 0.02878 to 0.16228). Even for parallel graphs, Top14 MAE reached 0.2447 in P1 and 0.2511 in P3, so adding sequential graphs alone cannot explain the gap. Limited simulator replays agreed, public-provider fitting errors were not uniformly worse, and Top14 retained more than 99.6% of the weight assigned within its candidate bank. The observed problem is that reconstructions fitted to individual providers' disclosures did not reliably predict the composed service under the new conditions. **No single cause has been established**; the changes were not isolated experimentally (Section 8.2).

Lesson for paper 2. A good pilot, provider-level fit or high ensemble weight coverage does not validate the end-to-end behavior of a hierarchical management or prediction method. The transferable recommendation is to specify the full procedure and its tuning rules in advance; evaluate across provider worlds, graph structures and SLA regimes on fresh cases; and report both prediction errors and downstream decisions by configuration. Controlled comparisons that change one factor at a time are needed before attributing failures to a particular algorithmic choice.

Variation across configurations. Tables 7 and 8 show the battery errors separately by provider world and composition graph.

Table 7: Expanded-battery mean absolute prediction error by provider world, averaged over four composition graphs (2,220 related prediction queries per world). FULL343 was executed in Phase II; original-style Top14 and Top1 were evaluated retrospectively in Phase III using the same Phase II reference.

Provider world	FULL343 MAE	Top14 MAE	Top1 MAE
P1	0.17326	0.21172	0.21149
P3	0.07169	0.12659	0.14299
P4	0.12073	0.12415	0.13237

Table 8: Expanded-battery mean absolute prediction error by composition graph, averaged over worlds P1, P3 and P4 (1,665 related prediction queries per graph). All three estimators use the same Phase II reference; Top14 and Top1 were evaluated retrospectively in Phase III.

Graph	FULL343 MAE	Top14 MAE	Top1 MAE
G_PAR	0.13586	0.20183	0.21773
G_PARSEQ	0.11016	0.14450	0.14318
G_SEQ	0.14624	0.14642	0.16105

Graph	FULL343 MAE	Top14 MAE	Top1 MAE
G_SEQPAR	0.09530	0.12387	0.12716

FULL343 had a lower MAE than Top14 in **11 of the 12 configurations**; Top14 had a lower MAE than Top1 in **8 of 12**. The overall difference between FULL343 and Top14 is strongly affected by one case, P3/G_PAR, which accounts for 49.3% of that aggregate MAE gap (Top14: 0.25108; FULL343: 0.06001). In P3/G_SEQ the ranking reverses (Top14: 0.08302; FULL343: 0.11633). Thus the aggregate ranking should not be interpreted as a consistent advantage in every setting.

Interpretation and limitations. The Top14 models captured 0.996404, 0.996771 and 0.998925 of the inferred joint-model weight in P1, P3 and P4, respectively. Within each finite candidate bank, the omitted-weight bound from Section 4.4 limits the corresponding difference from the full weighted prediction to 0.003596, 0.003229 and 0.001075. This small truncation bound does *not* bound error relative to the actual composed service: most of the inferred weight can still rest on inaccurate reconstructions.

These results also do not establish that using 343 joint models is intrinsically better than using 14. The original-style and V3b implementations differ in how provider candidates are generated, selected and weighted, even though their graph-simulation budgets match; their total reconstruction costs are not matched. Moreover, this comparison reused previously examined test cases and is not a new independent validation. The saved retrospective archive passed 316 numerical and SHA-256 consistency checks, including consistency of frozen predictions, model weights, selected support and reported errors. However, missing raw member curves and simulation ledgers prevent a complete independent replay. Section 8.2 investigates the documented pilot-to-battery changes in more detail.

Exact pilot-procedure transfer (additional retrospective feasibility check). Reproducing the original pilot established that its favorable result was computationally reproducible. Applying the same selection procedure to the expanded validation battery revealed a different limitation: the required model support exceeded the prescribed simulation budget in every additional provider world. Specifically, retaining 99.9% of the inferred joint-model probability weight required 14,267, 5,441 and 8,446 model combinations in P1, P3 and P4, respectively, whereas the fixed budget of 1,400 graph trajectories could not simulate each selected combination even once. Thus, even exact reproduction of a successful pilot does not establish the applicability of the method across heterogeneous configurations. This finding illustrates why validation must assess not only predictive accuracy, but also the feasibility and coverage of a method under diverse operating conditions.

8.2 Pilot–battery divergence tracing

8.3 Post-result I1/I2 pipeline development (historical Phase-6)

8.4 Distinct P2 public-induced-region comparison

8.5 Privileged search-deficit diagnosis

9. Uncertainty mathematics and cross-experiment interpretation

10. Threats to validity and reproducibility

11. Conclusions and falsifiable next questions

Appendix A. Mathematical and numerical interpretation

Appendix B. Archived source and audit provenance

Appendix C. Experimental chronology and method versions

Appendix D. Provisional literature references